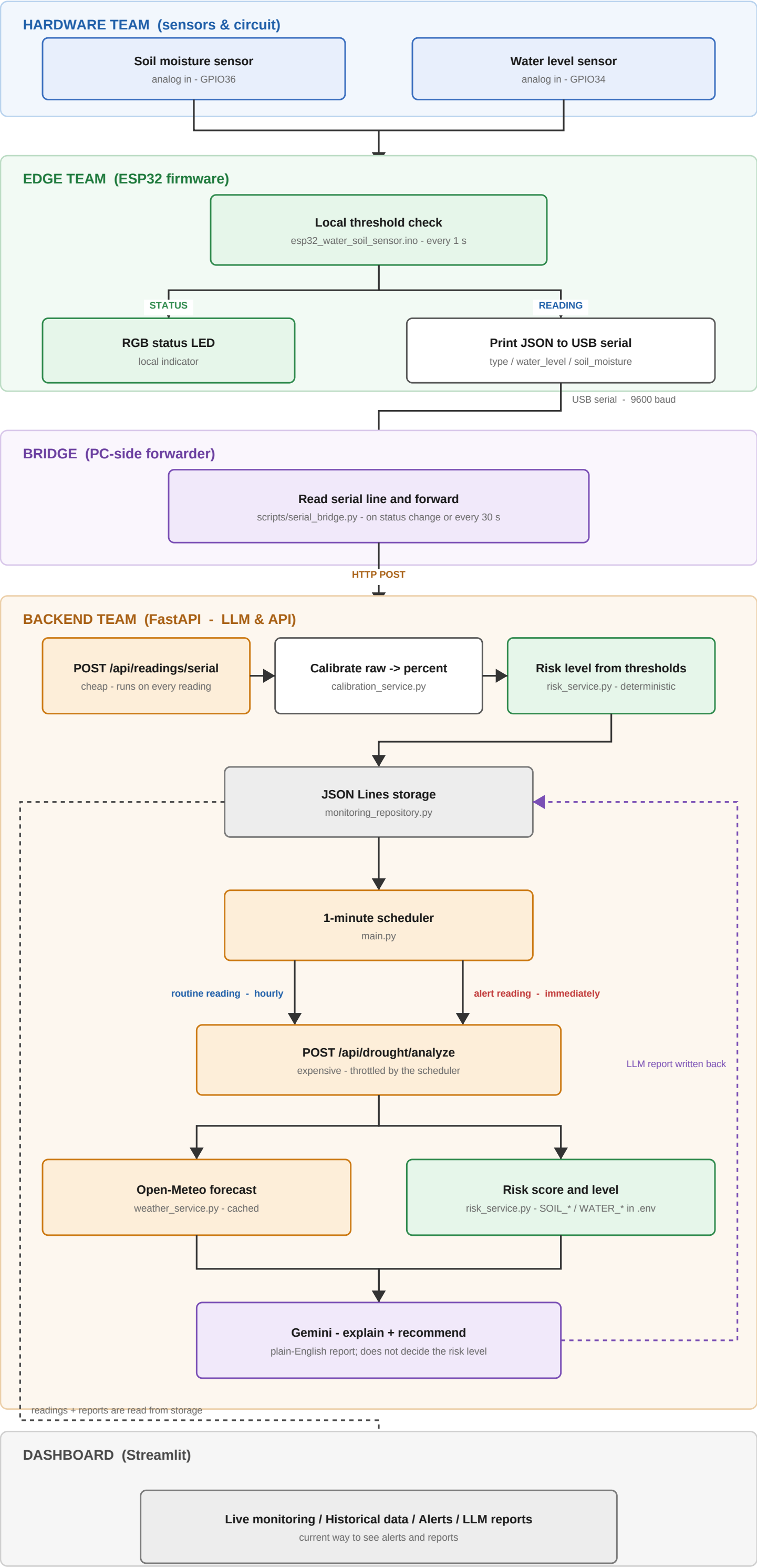


Drought Monitoring System

End-to-end data flow: sensors -> ESP32 -> serial bridge -> FastAPI backend -> dashboard



Notes

1. Edge device never calls the API

The ESP32 only prints JSON over USB serial. `scripts/serial_bridge.py`, running on a PC, is the only component that speaks HTTP to the backend. Wi-Fi was tried and dropped (see `environment-setup.md` and `implementation-rules-wifi.md`).

2. Two endpoints, two cost levels

`POST /api/readings/serial` is cheap and runs on every reading: it only calibrates and stores. `POST /api/drought/analyze` is the expensive call (weather + Gemini) and is throttled by the backend scheduler, not by the edge device.

3. The LLM does not decide the risk level

Gemini only explains and recommends in plain English. The deterministic risk score and level (`risk_service.py`, driven by the `SOIL_*` / `WATER_*` threshold values in `.env`) is what actually decides anomaly vs. normal.

4. Notification is dashboard-only for now

There is no app / LINE / Discord notification yet. The Streamlit dashboard is the current way to see alerts and reports.

5. Single integration point between teams

Hardware, edge, bridge and backend meet at exactly one contract: the JSON line printed on the serial port (`type / water_level / soil_moisture`). Each team can change its internals freely as long as that line keeps its shape.

Every connector in this diagram is drawn with horizontal and vertical segments only.